Xml66 Developer Guide 0.1.0

Chris Ahlstrom
(ahlstromcj@gmail.com)

March 23, 2026

Xml66 Logo

Contents

List of Figures

List of Tables

1 Introduction

The *Xml66* library reworks some of the fundamental code from the *Ardour* DAW project ([1]). It is basically a cut-down version of *libxml++* [NEED A CITE] that include just what is needed to read and process a MIDINAM [NEED A CITE] file.

Xml66 contains the following subdirectories of `src` and `include`, each of which holds modules in a namespace of the same name:

- `utfcpp`. Contains header files to support sanitizing UTF-8 data.
- `xml`. Provides C/C++ code reworked from the Ardour project.

In the sections that follow, the basics are described. At some point we will make the effort to add some *Dia* diagrams to make the relationships more clear.

1.1 Naming Conventions

Xml66 uses some conventions for naming things in this document.

- `$prefix`. The base location for installation of the application and its ancillary data files on *UNIX/Linux/BSD*:

  - `/usr/`
  - `/usr/local/`

- `$winprefix`. The base location for installation of the application and its ancillary data files on *Windows*.
  - `C:/Program Files/`
  - `C:/Program Files (x86)/`
- `$home`. The location of the user's configuration files. Not to be confused with `$HOME`, this is the standard location for configuration files. On a UNIX-style system, it would be `$HOME/.config/appname`. The files would be put into a `po` subdirectory here.
- `$winhome`. This location is different for *Windows*: `C:/Users/user/AppData/Local/PACKAGE`.

1.2 Future Work

- Hammer on this code in *Windows*.

2 XPath

This section discusses the main points of *XPath* as it applies to the partial coverage of XML supported by the *Xml66* library. See [6] for more complete information. An older specification is at [4].

XPath stands for *XML Path Language*. It uses a non-XML syntax for a flexible way of addressing different parts of an XML document. It can also test addressed nodes within a document to determine if they match a pattern. *XPath* models an XML document as a tree of nodes. There are different types of nodes, including *element nodes*, *attribute nodes*, and *text nodeso*. *XPath* defines a way to compute a string-value for each type of node. Some types of nodes also have names. XPath fully supports XML Namespaces.

The syntax of a path is somewhat complex, so we will distill it into the information needed to use the *Xml66* library.

2.1 Expr: expression

An *Expr* is the main syntactic construct of *Xpath*. It is evaluated, yielding one of these four types:

- **Node-set**. An unordered collection of nodes without duplicates.
- **Boolean**. A true or false value.
- **Number**. A floating-point number.
- **String**. A sequence of UCS characters.

2.2 Context

Expression evaluation occurs in a *Context*. The context consists of:

- **Node**. This is the "context"node.
- **Context position and size**. This is a pair of non-zero positive integers.

- **Set of variable bindings**. These are a mapping from variable names to variable values (each value is an object; see the previous section).
- **Function library**. These are a mapping from function names to functions.
- **Set of namespace declarations**. These consist of a mapping from prefixes to namespace URIs.

2.3 Location Path

The *Location Path* is a special case of an *Expr*, and is the most important construct. It has an unabbreviated syntax and an abbreviated syntax. Here a some of the former:

- **child::para**. Selects the *para* element children of the context node.
- **child::***. Selects all element children of the context node.
- **child::text()**. Selects all text node children of the context node.
- **child::node()**. Selects all the children of the context node, whatever their node type.
- **attribute::name**. Selects the name attribute of the context node.
- **attribute::***. Selects all the attributes of the context node.
- **descendant::para**. Selects the *para* element descendants of the context node.
- **descendant-or-self::para**. Selects the *para* element descendants of the context node and, if the context node is a *para* element, the context node as well.
- /. Selects the document root (which is always the parent of the document element).

There are many more. See [4]. In the abbreviated syntax, *child:* can be omitted. Here a some of the abbreviated syntax:

- **para**. Selects the *para* element children of the context node.
- *. Selects all element children of the context node.
- **text()**. Selects all text node children of the context node.
- **@**. An abbreviation for *attribute:*. Example: *para[@type="warning"]* is short for *child:para[attribute::type="*
- **@name**. Selects the name attribute of the context node.
- **@***. Selects all the attributes of the context node.
- **/doc/chapter[5]/section[2]**. Selects the second *section* of the fifth *chapter* of the *doc*.
- **chapter//para**. Selects the *para* element descendants of the *chapter* element children of the context node.
- **//para**. Selects all of the *para* element descendants of the document root, and hence selects all *para* element in the same document as the context node.
- **descendant-or-self::para**. Selects the *para* element descendants of the context node and, if the context node is a *para* element, the context node as well.
- /. Selects the document root (which is always the parent of the document element).
- //. Short for *descendant-or-self::node()*. By itself it is not a valid expression. It establishes an initial node sequence containing the root of the tree in which the context node is found, plus all nodes descended from this root. (The descendants of a node do *not* include *attribute* nodes or *namespace* nodes. A path expression that starts with "/" or "//" selects nodes starting from the context item's tree root; it is an absolute path expression.

The examples in the next sections all use files from *Ardour*'s test data. By default, the `xml66_tests` program shows minimal information and the success or failure status of the run. The `--verbose` option allows all the data to be shown, and the output is *long*.

2.3.1 Location Path Examples: Rosegarden

Here are some of the first items used in the `xml66_tests` program. They are used by first creating a document using an XML file, and calling the "find" function. In the *Rosegarden* XML document below, the hierarchy of nodes is "<rosegarden-data>", "<studio>", "<device>", then "<bank>".

basic_test_1():

```
xml66::XMLTree doc
(
   tests/data/RosegardenPatchFile.xml
);
xml66::SharedNodeListPtr nodeptrs
{
   doc.find("//bank[@name]")
};
```

"//bank[@name]" tells the parser to collect, from the top, all of the "<bank>" nodes that have the attribute "name". There are 8 of them. If "[@name]" is left out, then a 9th "bank" is found that does not have a name.

For each bank `b` in the node list, we get the bank name via `b->property("name")->value()`. For each bank, we then loop through the child nodes, which are "<program>" nodes, using the list `b->children()`. We can get their "id" and "name" values and show them or collect them.

basic_test_1b():

Another query using "//device[@name]" is similar. For each device, we can get their "id", "name", and "type" values and show them or collect them.

Other simple queries are "//librarian[@name]", "//controls/control[@name]"), and instrument[@name]".

basic_test_1c():

The "//controls/control[@name]" query retrieves all the "<controls>" and "<control>", and the test can show the names, type, and range of values for the controls.

The "//instrument[@name]" gets the "<instrument>" nodes and can show their id, and and type of instrument.

basic_test_2():

A more complex query is used in the test of the *Rosegarden* file:

```
xml66::SharedNodeListPtr nodeptrs
{
   doc.find
   (
      "/rosegarden-data/studio/device/bank/program[contains(@name, 'Latin')]"
   )
};
```

This absolute path navigates to the "<program>" nodes and collects only the ones having a "name" attribute that contains the word "Latin".

2.3.2 Location Path Examples: Ardour TestSession

These examples use an *Ardour* file from its test data.

basic_test_3():

```
xml66::XMLTree doc("tests/data/TestSession.ardour");
xml66::SharedNodeListPtr nodeptrs
{
   doc.find
   (
      "/Session/Sources/Source[contains(@captured-for, 'Guitar')]"
   )
};
```

This query finds all "<Sources>" where the "captured-for" attribute contains 'Guitar'."

basic_test_4():

The "//*[@id and @name]" query uses "//" to cover the whole document as the context node, "*" to select all element children, and "[@id and @name]" gets the nodes that have both "id" and "name" attributes.

2.3.3 Location Path Examples: Protools MIDINAM

These examples all use the file below, from *Ardour*'s test data. This is MIDINAM file, and is somewhat complex and contains some redundancy that bloats the file size significantly. It contains the following nodes of interest, and some not mentioned here:

- `MasterDeviceNames, Manufacturer, Model, CustomDeviceMode`. Specifies human-readable data about the device.
- `DeviceModeEnable` and `MIDICommands`. Enables modes of a device, and can specify a system-exclusive command, for example.
- `ChannelNameSetAssignments` and `ChannelNameSetAssign`. One example is Channel="1" NameSet="Name Set 1".
- `ChannelNameSet`. Name="Name Set 1" is an example.
- `AvailableForChannels`. This node, and the rest of the ones listed below, are included in a `ChannelNameSet` node.
- `AvailableChannel`. Channel="1" Available="true".
- `PatchBank`. Name="Piano".
- `PatchNameList`.
- `Patch`. Number="001" Name="Piano 1". There is a huge number of these nodes; they comprise the bulk of the data.
- `PatchMIDICommands`.

basic_test_5():

Here is the setup for the first query:

```
xml66::XMLTree doc
(
   "tests/data/ProtoolsPatchFile.midnam"
```

```
);
xml66::SharedNodeListPtr nodeptrs
{
   doc.find
   (
      "/MIDINameDocument/MasterDeviceNames/ChannelNameSet[@Name="
      "'Name Set 1']/PatchBank"
   )
};
```

This query get banks and patches for 'Name Set 1'. It starts by collecting all the "<PatchBank>" nodes. For each PatchBank, the following query, where p is the PatchBank, node is used.

```
xml66::SharedNodeListPtr nodeptrs { doc.find("//Patch[@Name]", p.get()) };
```

This call retrieves all of the "<Patch>" nodes in the PatchBank. The test program can then show them.

basic_test_6():

The query for this test is:

```
xml66::SharedNodeListPtr nodeptrs { doc.find("//@Value") };
```

This call retrieves *any* node that has an attribute named "Value". The program can then print the attribute name, which is always "Value", and the numeric value for that attribute. In the *Protools* example, 3318 values are found.

basic_test_7():

The query for this test is:

```
xml66::SharedNodeListPtr nodeptrs
{
   doc.find
   (
      "//ChannelNameSet[@Name = 'Name Set 1']"
      "//AvailableChannel[@Available = 'true']/@Channel"
   )
};
```

where this query is one long *C++* string. It gets 'Name Set 1', looks at all of the "<AvailableChannel>" nodes in it, and, if the "Available" attribute is "true", retrieves the value of the "Channel" attribute. In this file, channel 10 is marked false, and so is not counted.

2.4 Future Work

- Hammer on this documentation.

3 Summary

Contact: If you have ideas about *Xml66* or a bug report, please email us (at mailto:ahlstromcj@gmail.com).

4 References

The *Xml66* references list.

References

[1] Authors? *Ardour: Record, Edit, and Mix on Linux, macOS and Windows* https://ardour.org/ 20??-2026

[2] Authors? *libxml++: C++ interface to libxml2 and XML files.* https://github.com/libxmlplusplus/libxmlplusplus 2019-2026.

[3] Chris Ahlstrom. *A reboot of the Seq24 project as "Seq66".* https://github.com/ahlstromcj/seq66/. 2015-2024.

[4] w3.org *XML Path Language (XPath) Version 1.0* https://www.w3.org/TR/1999/REC-xpath-19991116/#section-Introduction 1999.

[5] w3.org *XML Path Language (XPath) Version 3.1* https://www.w3.org/TR/xpath-31 2017.

[6] Mozilla.org *Xpath: XML Path Language.* https://developer.mozilla.org/en-US/docs/Web/XML/XPath 2025.